Synechron L2 Interview – Master Q&A

Synechron L2 • Senior DevOps / Platform Engineering / SRE / Banking • Interview prep

A. Profile & Role Fit

Q: Tell me about yourself.

Spoken answer: I have 11+ years across infrastructure, middleware, DevOps, platform engineering and SRE. I moved from application and middleware operations into automation, CI/CD, cloud and Kubernetes. Most recently, on Microsoft Azure Synapse Analytics, I worked on production reliability, P1/P2 issues, SLI/SLOs, observability, alert optimisation and platform health. My core strengths are AWS/Azure, Kubernetes, Terraform, CI/CD, Python/Bash and production operations.

Q: Have you worked in DevOps?

Spoken answer: Yes. I have worked with Git, Jenkins, GitHub Actions, build and artifact management, automated deployments, Terraform, Docker/Kubernetes and production support. I focus on repeatable, traceable deployments and reducing manual operational effort.

Q: Have you worked in platform engineering?

Spoken answer: My experience strongly overlaps with platform engineering. I have supported shared Kubernetes and cloud platforms, standardised deployment patterns, automation, observability, access and operational runbooks. My approach is to provide secure, reusable and observable platform capabilities for application teams.

Q: Have you worked as an SRE?

Spoken answer: Yes. My recent work was strongly SRE-oriented: production reliability, P1/P2 incidents, SLI/SLO definition, monitoring and alert tuning, capacity and platform health, automation and post-incident improvement. I use reliability principles such as reducing toil, actionable alerts and measurable service health.

Q: Have you worked in banking?

Spoken answer: I have worked in enterprise production environments with controlled changes, security requirements, operational governance and audit-oriented processes. For a banking environment I would apply the same disciplined approach: approvals, segregation of duties, traceability, implementation and rollback plans, evidence and post-release validation.

B. DevOps & Release Engineering

Q: Describe your CI/CD approach.

Spoken answer: Git is the source of truth. I build and test, run quality and security checks, create an immutable artifact or image, and promote the same artifact through environments. Production requires the appropriate approvals and change record. After deployment I validate smoke tests, logs, metrics and business health. My principle is build once and promote.

Q: How do you make a production deployment safe?

Spoken answer: I define scope, dependencies, readiness criteria, implementation steps, rollback criteria, monitoring and validation before deployment. I verify approvals and support coverage, execute in a controlled window, run smoke tests, watch technical and business metrics, and capture evidence before closure.

Q: What is your rollback strategy?

Spoken answer: Rollback is designed before deployment. I define objective triggers, keep the previous known-good version available, verify database/config compatibility and document the exact rollback mechanism. If criteria are met, I stop rollout, rollback or mitigate, validate recovery, communicate and complete RCA.

Q: Blue-green vs canary vs rolling?

Spoken answer: Blue-green uses two environments and switches traffic, giving a fast rollback path. Canary sends a small percentage to the new version and increases gradually based on metrics. Rolling replaces instances or pods incrementally. I choose based on risk, traffic control and rollback requirements.

C. Kubernetes / Platform Engineering

Q: Explain Kubernetes production architecture.

Spoken answer: The control plane manages cluster state and scheduling; worker nodes provide compute. Deployments, StatefulSets, Jobs and DaemonSets manage workloads. Services provide stable discovery, Ingress handles HTTP/HTTPS entry, ConfigMaps and Secrets provide configuration, and observability captures metrics, logs and events. Production also needs RBAC, NetworkPolicies, requests/limits, probes, PDBs and capacity planning.

Q: A pod is Pending. What do you check?

Spoken answer: I start with kubectl describe pod and events. I check node capacity, resource requests, taints/tolerations, node selectors or affinity, namespace quotas, PVC binding and IP/subnet capacity. I identify the scheduler constraint rather than restarting the pod blindly.

Q: CrashLoopBackOff?

Spoken answer: I check current and previous logs, describe the pod and inspect events. Then I verify commands, ConfigMaps, Secrets, dependencies, probes and resources. If it is OOMKilled I investigate memory usage and limits. I also check recent deployment/configuration changes and rollback if the release caused the incident.

Q: How do pods communicate securely?

Spoken answer: Services provide discovery and NetworkPolicies control traffic. For sensitive workloads I prefer default-deny and explicit allow rules by namespace, pod labels and ports. TLS or a service mesh can add encryption and identity when required.

D. AWS / Azure

Q: Explain EKS networking.

Spoken answer: EKS uses AWS VPC networking. With the AWS VPC CNI, pods receive VPC IPs through ENIs, so subnet capacity matters. I check VPC/subnet CIDRs, routes, security groups, NACLs, NAT/Internet Gateway, DNS and load balancers. I plan address space for nodes, pods and growth.

Q: How would you size about 1,500 IPs?

Spoken answer: A /21 has 2,048 total IPv4 addresses and a /20 has 4,096. AWS reserves five addresses in each subnet, and EKS capacity also depends on ENI and pod-density limits. I therefore calculate actual node/pod demand and future headroom instead of selecting the CIDR from one number alone.

Q: Active-active across AWS regions?

Spoken answer: I deploy equivalent stacks in multiple regions, use health-based traffic steering such as Global Accelerator or Route 53, reproduce infrastructure with Terraform, and design data replication for the required RTO/RPO. I monitor both regions and regularly test failover.

Q: EKS to RDS connectivity?

Spoken answer: I keep RDS private where possible. I validate VPC routing and DNS, and allow the DB port in the RDS security group from the workload security group rather than a broad CIDR. I also validate credentials or IAM authentication, TLS and connection limits.

E. Terraform & Automation

Q: How do you use Terraform in production?

Spoken answer: Terraform stays in Git with reusable modules. CI runs fmt, validate and plan; changes go through review and controlled apply. I use remote state with locking and access controls. I avoid unmanaged console changes and reconcile legitimate emergency changes back into code.

Q: How do you handle drift?

Spoken answer: I run terraform plan, determine whether the real-world change is expected, then update code if it is legitimate or restore declared state if it is not. I review destructive changes before applying in production.

Q: Give an automation example.

Spoken answer: I automated repetitive post-deployment work such as cache clearing and cookie/configuration updates using Rundeck and REST API or curl instead of manual UI steps. The result was repeatability and less toil. I apply the same principle with Python or Bash.

F. SRE / Observability / Incidents

Q: SLI, SLO and SLA?

Spoken answer: An SLI is the measured indicator, such as successful requests or latency. An SLO is the internal reliability target, such as 99.9% success. An SLA is a contractual commitment. I use SLOs and error budgets to make reliability measurable and balance reliability with delivery.

Q: Golden Signals?

Spoken answer: Latency, traffic, errors and saturation. I add service-specific and business metrics where needed.

Q: How do you reduce alert noise?

Spoken answer: I remove duplicate and non-actionable alerts, tune thresholds and evaluation windows, use symptom-based alerts, and improve routing/suppression. In my recent environment I worked on alert optimisation and reduced alert noise by about 40%.

Q: How do you handle P1?

Spoken answer: I assess impact, establish an incident owner and communication channel, triage using metrics/logs/traces and recent changes, mitigate quickly, communicate status, validate recovery, preserve evidence, complete RCA and implement preventive actions.

G. Banking / Governance

Q: UAT approval is missing. Release?

Spoken answer: If it is mandatory, I do not bypass it. I identify the missing approval, escalate to the accountable owner and use only the organisation's approved exception process. Production proceeds only when the required control or formally approved exception exists.

Q: What is ServiceNow change management?

Spoken answer: The change captures scope, risk, implementation, validation, rollback, schedule, affected CIs and approvals. During execution I update status and evidence, link incidents/problems where relevant, and close only after validation and required documentation.

Q: Banking production release checklist?

Spoken answer: Approved change, tested build, exact version, UAT/security approvals, dependency readiness, DB/config/certificates/secrets, access, monitoring, implementation and rollback, smoke tests, support coverage, communications and audit evidence.

Q: How do you balance speed and controls?

Spoken answer: I automate controls such as testing, security scans, evidence and deployment checks while preserving mandatory approvals and segregation of duties. The goal is compliant delivery that is faster and repeatable.

H. Leadership Scenarios

Q: Dev says it works, production fails. What do you do?

Spoken answer: I avoid debating ownership. I establish impact, compare production with the last known-good state, inspect logs/metrics/config/dependencies/recent changes, mitigate first and involve the correct owner based on evidence. After recovery I document RCA and preventive actions.

Q: Two teams want the same deployment window.

Spoken answer: I make dependencies and business impact visible, check the release calendar, see whether releases can be decoupled, propose a lower-risk sequence and escalate with facts if needed.

Q: Why are you suitable?

Spoken answer: I combine production operations, SRE, DevOps, cloud, Kubernetes, automation and controlled release experience. I can work technically and coordinate across teams, challenge readiness gaps, manage incidents and make production changes traceable and supportable.

Q: Why Synechron?

Spoken answer: The role combines enterprise engineering, cloud, DevOps/SRE and structured delivery, which aligns well with my experience. I can contribute technically while taking ownership of predictable and controlled production delivery.